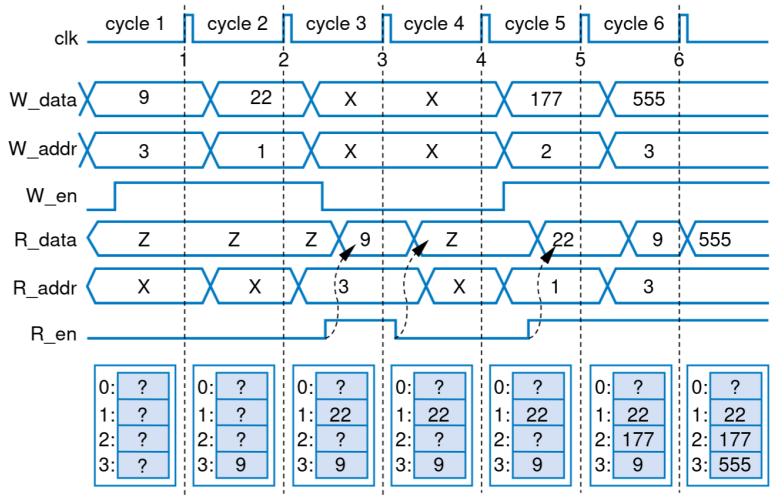
1 Lesson 23-10-26

Last lesson we looked at *register files*. Specifically, we landed on an implementation which used 2 solutions we hadn't seen before:

- **Amplifiers** or *buffers*, purely electrical devices which we used to boost the signal of the data input on the path to all registers;
- **Tri-state drivers**, which we used to select the register that drives the output data bus (leaving the others in high impedance).

1.0.1 Register file timing

Let's quickly see a graph of timing constants for register files. We see that operations are clock-synchronized, with data inputs and outputs (and corresponding address lines) having to be stable at clock rising edges.



1.1 RTL design

So far we focused on behavioral and maybe dataflow designs of combinatorial and sequential devices. Now we want to move on to **RTL** (*Register Transfer Level*) designs, which really sit at an higher level of abstraction.

We already defined embedded systems as digital electronic systems focused not on general purpose but on specific applications.

1.1.1 Metrics

Among the design metrics we are optimizing for there's an obvious design goal:

- Construct an implementation with desired functionality.

The key design challenge is to simultaneously optimize numerous *design metrics*. A **design metric** is a measurable feature of a system's implementation.

Some common metrics are:

- **Unit cost**: the monetary cost of manufacturing each copy of the system, excluding NRE cost;

- **NRE cost** (*Non-Recurring Engineering cost*): the one-time monetary cost of designing the system;
- **Size**: the physical space required by the system;
- **Performance**: the execution time or throughput of the system;
- **Power**: the amount of power consumed by the system;
- **Flexibility**: the ability to change the functionality of the system without incurring heavy NRE costs;
- **Time-to-prototype**: the time needed to build a working version of the system;
- **Time-to-market**: the time required to develop a system to the point that it can be released and sold to customers;
- **Maintainability**: the ability to modify the system after its initial release;
- Other metrics include *correctness*, *safety*, ...

1.1.2 Technologies

Optimizing means choosing techonologies, so let's go over the 3 main types:

- **VLSI** (*Very Large Scale Integration*), also known as full custom, all layers are optimized for an embedded system's particular digital implementation. This means placing and sizing transistors, routing wires, ...

  The benefits are excellent performance, small size, low power, at the cost of high NRE cost and time-to-market;

- **ASIC**s (*Application Specific Integrated Circuits*), lower layers are fully or partially built, and designers are left with routing of wires and maybe placing some blocks.

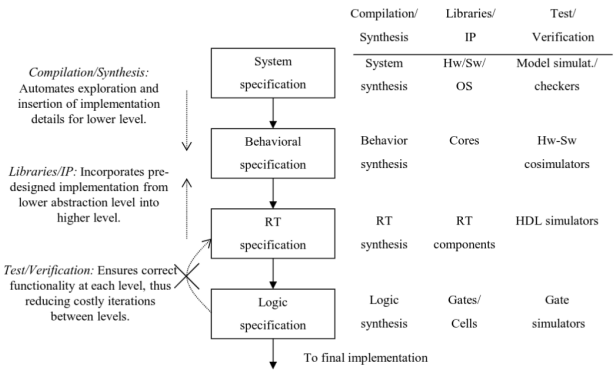
  The benefits are good performance, good size, less NRE cost than a full-custom solution. Time-to-market and costs are still pretty high, though;

- **PLDs** (*Programmable Logic Devices*): all layers already exist, designers can purchase an IC, and the connections on the IC are either created or destroyed to implement desired functionality (usually via a software programmer). *Field-Programmable Gate Array* (**FPGA**s) are very popular PLDs.

  The benefits are low NRE costs, almost instant IC availability, while the drawback is that PLDs are bigger, expensive (per-unit) and, power hungry. hungry, slower

1.1.3 Design flow

What we are interested in in this course is the design technology, that is the manner in which we convert our concept of desired system functionality into an implementation, rather than technology in it of itself. We know, for instance, that our tools allow us to easily convert our designs to FPGA programmer instructions.



1.1.4 Recap

In the previous lessons we saw:

- **Combinatorial** networks, so ways to map equations and truth tables to logic;
- **Sequential** networks, so FSMs, registers, etc...
- **Datapath** components based on combinatorial and sequential designs which allowed simple actual elaboration on data.

We know get to the point of **RTL** design, where we intend to:

- Capture circuit *behavior* via high level FSMs;
- Convert these FSMs to circuits via the **controller** + **datapath** design model (where a controller controls a datapath forming a **processor**).

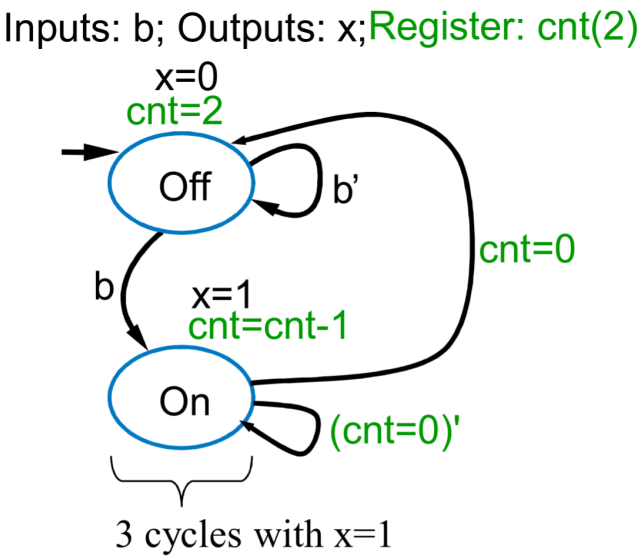
1.2 High-Level State Machines

Some behaviors are too complex for equations, truth tables, or simple FSMs. This is where we extend FSMs with features such as *state* variables, forming **higher-level** SMs.

More specifically, the **HLSM** (*High-Level State Machine*) model extends FSM with:

- Multi-bit input/output;
- Local storage (state variables);
- Arithmetic operations on those state variables.

Let's take a simple example of a variable counter. For an N-counter, we could design an $N + 1$ state FSM, but that becomes cumbersome fast. A better approach is to use an FSM governing a register.



Note that this can be interpreted as an FSM where the status register is *"extended"* by the bits of the counter register.

